Software Requirements Specification

For

VisEntrixia

Local Tourist Day-Visit Planner and Information System for Akkaraipattu

Version 1.1 – Final SRS

Prepared by Ahamed BA (E2410805)

University of Moratuwa

06 September 2026

Table of Contents

List of Tables iv

List of Figures iv

Revision History iv

1.Introduction 1

1.1 Purpose 1

1.2 Document Conventions 1

1.3 Intended Audience and Reading Suggestions 1

1.4 Product Scope 2

1.5 References 3

2. Overall Description 4

2.1 Product Perspective 4

2.2 Product Functions 4

2.3 User Classes and Characteristics 5

2.4 Operating Environment 5

2.5 Design and Implementation Constraints 5

2.6 User Documentation 6

2.7 Assumptions and Dependencies 6

2.8 Stakeholder Identification 7

3. External Interface Requirements 8

3.1 User Interfaces 8

3.2 Hardware Interfaces 9

3.3 Software Interfaces 9

3.4 Communications Interfaces 9

4. System Features 10

4.1 Attraction Browsing, Search, and Filter 10

4.1.1 Description and Priority 10

4.1.2 Stimulus/Response Sequences 10

4.1.3 Functional Requirements 10

4.2 Detailed Information and Map Integration 10

4.2.1 Description and Priority 10

4.2.2 Stimulus/Response Sequences 10

4.2.3 Functional Requirements 11
4.3 One-Day Itinerary Planning 11
4.3.1 Description and Priority 11
4.3.2 Stimulus/Response Sequences 11
4.3.3 Functional Requirements 11
4.4 User Authentication 12
4.4.1 Description and Priority 12
4.4.2 Stimulus/Response Sequences 12
4.4.3 Functional Requirements 12
4.5 Administrative Content Management (CRUD) 13
4.5.1 Description and Priority 13
4.5.2 Stimulus/Response Sequences 13
4.5.3 Functional Requirements 13
5. Other Nonfunctional Requirements 14
5.1 Performance Requirements 14
5.2 Safety Requirements 14
5.3 Security Requirements 15
5.4 Software Quality Attributes 16
5.5 Business Rules 17
6. Other Requirements 18
Appendix A: Glossary 18
Appendix B: Analysis Models 19
B.1 Entity Relationship Diagram 19
B.2 Use Case Diagram 20
Appendix C: To Be Determined List 21

List of Tables

Table 1 Stakeholder Requirements 7
Table 2 Software Interfaces 9
Table 3 Glossary of Terms and Abbreviations 18
Table 4 To Be Determined List 21

List of Figures

Figure i Entity-Relationship (ER) Diagram 19
Figure ii Use Case Diagram 20

Revision History

Name	Date	Reason for Changes	Version
Ahamed BA	19 August 2026	SRS drafted	0.1
Ahamed BA	06 September 2026	SRS Final	1.1

1.Introduction

The Akkaraipattu area is considered to be a very significant cultural and historical landmark of the Eastern Province of Sri Lanka as it is within 25km radius from my home place in Ampara District. The region is known for its lush agricultural scenes and the peaceful coexistence of Muslim and Tamil people [1]. In the past, it was an important trade center and now it has large mosques, kovils and it is very close to various natural spots such as lagoons, reservoirs and national parks. Even this enormous potential is not attracting much tourist attention in Akkaraipattu. The major problem on the way to visitor engagement is the lack of a reliable and central digital information system. There are currently no up-to-date information, such as details about local attractions, historical sites, logistical needs, which are generally not available online or are hard to verify. This information deficit can make it difficult for potential tourists to find hidden gems in the region and effectively plan a trip. Therefore, the socio-cultural and natural richness of the area is not used optimally in terms of economic and community development. This project directly targets this problem by its objective to create a well organized digital information system. The aim is to fill this gap, to make Akkaraipattu more accessible, to encourage sustainable tourism, and to bring to life the potential of Akkaraipattu for tourists and the local community.

1.1 Purpose

This is a Software Requirements Specification (SRS) of a web based Local Tourist Day-Visit Planner and Information System for Akkaraipattu area. This is the second submission for ITE2953 Programming Group Project, extending the work done with the Project proposal submitted earlier in the semester to specify the problem statement, aim, objectives, scope, stakeholder requirements and technology stack. It is this SRS that turns the above proposal into a detailed specification which will be used for designing, implementing and testing the system (version 1.0).

1.2 Document Conventions

This SRS follows the IEEE Std 830-1998 standard. Functional requirements are tagged as **FR-XXX**, non-functional requirements as **NFR-XXX**, and to-be-determined items as **TBD-XXX**. Requirement priorities are classified as High, Medium, or Low. Each requirement is uniquely numbered and verifiable through testing or inspection [1].

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, project managers, testers, stakeholders, and documentation writers. Developers should focus on Sections 2, 3, 4, and 5 for implementation details. Project managers should review Sections 1, 2, and Appendix C for project planning.

Testers should study Sections 2, 3, 4, and 5 for verification. All readers should begin with Section 1 for project context and refer to Appendix A for terminology definitions.

1.4 Product Scope

In order to address the challenge of unavailability and reliability of information and planning regarding tourism in the Akkaraipattu region of the Eastern Province of Sri Lanka a new centralized information and planning system, known as the **VisEntrixia** was developed and launched using the web platform. The system will let tourists browse, search, filter and sort places of interest within a 25 km radius of Akkaraipattu; view detailed information for each place (description, category, opening hours, entrance fees, travel tips and distance); locate places on an integrated map; and assemble a personalized one-day itinerary from the places they select. The place data is managed by a system administrator using a secure, authenticated interface that allows for create, read, update and delete (CRUD) operations. The system won't provide online booking and payment functions, or make transportation or restaurant reservations. The system provides optional user accounts for tourists who wish to save their itineraries. Tourists can browse places, view details, use maps, and build a session-based itinerary without logging in. Only the save itinerary feature requires authentication. Only information and day-trip plans can be provided. The project aims to close this information gap and will positively influence the local economy in the region, cultural and heritage tourism and visitors to the area.

1.5 References

[1] D. B. McGilvray, "Caste and Matriclan Structure in Eastern Sri Lanka: A Preliminary Report on Fieldwork in Akkaraipattu," University of Peradeniya.

[2] IEEE , "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998. [Online]. Available: https://standards.ieee.org/ieee/830/1402/. [Accessed 20 Augest 2026].

[3] V. Subramanian, Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node, Berkeley, CA: Apress, 2019.

[4] Google Developers, "Google Maps Platform Documentation," Google LLC, [Online]. Available: https://developers.google.com/maps. [Accessed 20 Augest 2026].

2. Overall Description

2.1 Product Perspective

VisEntrixia is a new, self-contained web application, which is not a follow-on to an existing system, and has no legacy components to integrate with. It is built on a typical three-layer structure: the front-end part is implemented using React.js as an S.P.A., the back-end is provided using Node.js/Express.js and exposes a REST API, and the place records are stored in a MongoDB database. The frontend also communicates with the Leaflet.js mapping library using OpenStreetMap tiles to render place locations and enable map-based navigation. These components and the interaction of the tourist users.

2.2 Product Functions

The tourism information system 'VisEntrixia' offers various functions to help users explore and plan a visit to the Akkaraipattu region. The main activities of the product are:

- **Attraction Browsing:** Presenting a complete list of the attractions in the area within 25 km of Akkaraipattu.
- **Search and Categorization:** Give users the ability to search, filter and sort locations by specific categories (Religious, Nature, Heritage, Cultural, Culinary).
- **Detailed Information Display:** Offers detailed information for each location, such as descriptions, hours, travel tips, and distances from a central point.
- **Map Integration:** Offering an integrated map interface to indicate the exact geographical location of each place for easier navigation.
- **Itinerary Planning:** Allow tourists to develop a personalized one-day itinerary by choosing and arranging their favourite locations. Tourists can build and view itineraries without logging in. However, to save an itinerary for future reference, they must register and log in.
- **Saved Itineraries:** Allow logged-in tourists to save, view, and delete their previously generated itineraries.
- **Content Management (CRUD):** Providing a secure back end interface for administrators to Create, Read, Update and Delete place data in the database.

2.3 User Classes and Characteristics

The system is designed for two user classes with different requirements and privileges:

Tourist User

Visitors to the Akkaraipattu and wider Ampara area, ranging from first-time day tourists to families, budget travellers, solo travellers and nature/adventure explorers. No particular technical expertise is assumed; the interface must be usable by casual, non-technical visitors on both desktop and mobile browsers.

The Tourist User operates in two modes:

1. No Login Required: Tourists can browse places, search, filter, sort, view detailed information, use the interactive map, and build a temporary session-based one-day itinerary all without creating an account.

2. Logged In: Tourists who choose to create an account and log in gain an additional feature: they can **save their generated itineraries** to their account for future reference and view their previously saved itineraries. Registration is optional and only required for the save feature.

System Administrator

A single content-management role responsible for keeping place data accurate and current — adding new places, editing existing details such as descriptions or opening hours, and removing outdated entries. This user requires an authenticated login and is assumed to have basic familiarity with web-based admin interfaces. This is the most privileged, but least frequently used, class.

2.4 Operating Environment

VisEntrixia is a browser-based web application that will run on any modern web browser with internet access – such as current versions of Chrome, Firefox, Safari and Edge. It is coded in the Node.js runtime and uses the Express.js framework, plus a MongoDB database and may be deployed anywhere that supports a Node runtime.

2.5 Design and Implementation Constraints

The design and development of the VisEntrixia tourism information system is subject to various technological and functional limitations that guarantee feasibility and concentration. These constraints include:

The design and development of the VisEntrixia tourism information system is subject to various technological and functional limitations that guarantee feasibility and concentration:

Technology Stack Mandate: The web application shall be built using the MERN stack (MongoDB, Express.js, React.js, Node.js) with JavaScript (ES6+) .

Frontend Standards: User interface development is limited to React.js with JavaScript (JSX), using HTML5 and CSS3.

Mapping Library: Map display and geolocation features shall be implemented using Leaflet with OpenStreetMap tiles (no paid API key required).

Development and Testing Tools: All tools used for development are limited to Visual Studio Code for coding, Postman for API testing, MongoDB Atlas for database visualization, and Git (via GitHub) for version control.

Functional Limitations (Scope Restrictions): The system can only be used as an information and itinerary planning tool. It will specifically have no modules for online booking (such as hotels, safaris etc.) and will not accept any payment processing.

Geographical Data Constraint: The system contains data of only places of interest within a 25 km radius of Akkaraipattu.

2.6 User Documentation

TBD — End-user documentation (such as an in-app help page, short user guide or on-screen tooltips) will accompany the delivered system, although this was not specified. To be confirmed before final submission. This is due to the very simple interface of the system, it is good to have a brief 'how to' for a first time tourist user as well as a brief note on how to operate it as an admin.

2.7 Assumptions and Dependencies

- Mapping feature relies on the continuous availability of Leaflet and OpenStreetMap tile servers for map rendering and location display.

- It is assumed that the data (such as descriptions, categories, distances, opening times, fees) are manually curated and entered by the administrator and that there is no dependence on any third party tourism data feed.

- Tourists (whether logged in or not) can browse places, view details, use maps, and build a session-based itinerary. Only the save itinerary feature requires registration and login.

- The stakeholder requirements and preliminary place list collected in the first submission is assumed to be a good approximation of the needs of real tourists to inform design.

2.8 Stakeholder Identification

The real user requirements and the validation of the system requirements will be carried out by four colleagues of the project group as users who will be consulted as stakeholders. They will pretend to be end-users to give feedback on what kind of information and features would be useful to a tourist coming to this area.

Table 1 Stakeholder Requirements

Stakeholder	Role	Key requirement identified
Salgado MLS	Family Traveller	Rating Filter, Child Activities, Entrance fee, Opening Hours, Safety Information, Parking, 1 day itinerary suggestion – family friendly.
Palihawadana PART	Budget Traveller	Budget Traveller Entry & transport fees, free attractions, low-cost food, public transport routes, filters by cost/distance, estimated total cost & travel time.
Ranasinghe RAKS	Adventure& Nature Explorer	Route distance & completion time, real time weather alerts, route status (open/blocked), restrooms, water stations, camping sites, entry & guide fees.
Induwara HDL	Local Community Member	Hidden cultural spots, local food tips, dress codes, transportation info, traffic/closure alerts, opening hours, filters (distance/open status), offline saving, day plan builder.

3. External Interface Requirements

3.1 User Interfaces

The user-friendly front end of the VisEntrixia web application will be created with React.js and JavaScript (JSX), using HTML5 and CSS3 for markup and styling. The system's graphical user interfaces (GUI) are logically divided based on two user classes: Tourist Users and System Administrators.

Tourist Interface (No Login Required)

- Open Access: All tourists can use the platform without any registration or login process.
- Search & Filter: Quick search and filter controls enabling tourists to browse places within a 25km radius. Places can be categorized by Religious, Nature, Heritage, Cultural, and Culinary.
- Detailed View & Map: Access to detailed place information such as descriptions, opening hours, travel tips, and an embedded Leaflet map showing the exact location.
- Itinerary Builder: Tourists can add, remove, and reorder attractions to build a one-day itinerary. This is held temporarily in the browser session.

Tourist Interface (Login Required for Save Feature)

- Save Itinerary: A "Save Itinerary" button is displayed. If clicked by an unregistered tourist, the system prompts them to register or log in. Once logged in, the tourist can save the current itinerary to their account.
- Saved Itineraries List: A view showing all previously saved itineraries with their creation dates and place lists.

System Administrator Interface

- Admin Login Portal: A secure and restricted login screen exclusively for system administrators.

- Content Management Dashboard: A comprehensive backend dashboard with the ability to perform all CRUD operations. The interface allows administrators to add places, edit descriptions and timings, and delete outdated place data.

3.2 Hardware Interfaces

VisEntrixia has no dedicated hardware interfaces. It relies only on the standard input (touchscreen, mouse, keyboard) and output (display) capabilities of the desktop or mobile device running the browser, and on that device's network hardware for internet connectivity.

3.3 Software Interfaces

Table 2 Software Interfaces

Component	Technology	Purpose
Frontend	React.js with JavaScript (JSX, HTML5, CSS3)	Renders the tourist and admin interfaces and consumes the backend REST API
Backend	Node.js with Express.js	Exposes REST endpoints for place data retrieval and admin CRUD operations.
Database	MongoDB	Stores place records and administrator credentials.
Mapping	Leaflet.js with OpenStreetMap tiles	Provides interactive map rendering and place-location display (no paid API key required)
Test Tools	Visual Studio Code, Postman, MongoDB Compass, Git/GitHub	Development, API testing, database inspection and version control.

3.4 Communications Interfaces

The frontend and backend communicate over HTTP/HTTPS using a JSON-based REST API. The frontend separately communicates with OpenStreetMap tile servers over HTTPS to retrieve map tiles for the Leaflet map display. User authentication is handled via JSON Web Tokens (JWT) stored in HTTP-only cookies for secure session management.

4. System Features

4.1 Attraction Browsing, Search, and Filter

4.1.1 Description and Priority

Users can browse a list of attractions within a 25km radius, search by name, filter by category (Religious, Nature, Heritage, Cultural, Culinary), and sort results.
Priority: HIGH

4.1.2 Stimulus/Response Sequences

- Browse: User loads the page - System displays all attractions.
- Filter: User selects a category - System shows only that category.
- Search: User types a keyword - System shows matching attractions.
- Sort: User selects sort option -System reorders the list.

4.1.3 Functional Requirements

- FR-01: The system shall display all attractions with name, category, brief description, and distance.
- FR-02: The system shall allow filtering by Religious, Nature, Heritage, Cultural, and Culinary.
- FR-03: The system shall allow searching by name or description.
- FR-04: The system shall support sorting by name (A-Z) and distance (nearest first).

4.2 Detailed Information and Map Integration

4.2.1 Description and Priority

Users can view complete details of any attraction and see its exact location on an integrated Leaflet map with directions.

Priority: HIGH

4.2.2 Stimulus/Response Sequences

- View Details: User clicks an attraction - System shows full description, hours, fees, tips, and distance.
- View on Map: User clicks "View on Map" -System centers the map on that location.

- Get Directions: User clicks "Directions" - System opens OpenStreetMap routing with the route.

4.2.3 Functional Requirements

- FR-05: The system shall show detailed information: description, opening hours, entrance fees, travel tips, and distance.
- FR-06: The system shall integrate Leaflet with OpenStreetMap to show the exact location of each attraction.
- FR-07: The system shall provide a "Directions" feature using OpenStreetMap routing to navigate from Akkaraipattu town.

4.3 One-Day Itinerary Planning

4.3.1 Description and Priority

Tourists can create a simple one-day plan by adding, removing, and reordering selected attractions. Registration is NOT required to build a session-based itinerary. However, to save the itinerary for future reference, tourists must register and log in.

Priority: HIGH

4.3.2 Stimulus/Response Sequences

- Add: User clicks "Add to Itinerary" - System adds the place and updates the counter (available without login).
- View: User goes to itinerary page - System shows all selected places in order (available without login).
- Remove: User clicks "Remove" - System deletes it from the plan (available without login).
- Reorder: User drags and drops - System updates the order (available without login).
- Save: User clicks "Save Itinerary" - If not logged in, system prompts to login/register. If logged in, system saves to account.

4.3.3 Functional Requirements

- FR-08: The system shall allow all tourists to add attractions to a session-based itinerary without logging in.
- FR-09: The system shall prevent duplicate entries in the same itinerary.
- FR-10: The system shall allow tourists to remove and reorder (drag-and-drop) places.

- FR-11: The system shall allow logged-in tourists to save their itinerary permanently in the database, linked to their registered account.
- FR-12: The system shall display a counter showing the number of places in the itinerary.
- FR-13: If an unregistered tourist attempts to save an itinerary, the system shall display a prompt encouraging registration or login.

4.4 User Authentication

4.4.1 Description and Priority

Secure registration and login for Tourist Users (to save itineraries) and System Administrators (to access the admin panel). Tourists do NOT need to log in to browse, search, view details, use maps, or build a session-based itinerary.

Priority: HIGH

4.4.2 Stimulus/Response Sequences

- Register: User submits name, email, password - System creates an account and redirects to login.
- Login: User enters credentials - System validates and creates a JWT session.
- Admin Login: Admin logs in - System validates and redirects to the admin dashboard.
- Logout: User clicks logout - System invalidates the session.

4.4.3 Functional Requirements

- FR-14: The system shall provide registration (name, unique email, secure password).
- FR-15: The system shall hash passwords using bcrypt before storage.
- FR-16: The system shall provide login for both Tourist Users and Administrators using JWT.
- FR-17: The system shall expire sessions after 60 minutes of inactivity.
- FR-18: The system shall lock accounts after 5 failed login attempts (15-minute lockout).
- FR-19: The system shall allow tourists (unregistered) to browse, search, view details, use maps, and build session-based itineraries without authentication.
- FR-20: The system shall require authentication only for the Save Itinerary feature.

4.5 Administrative Content Management (CRUD)

4.5.1 Description and Priority

A secure admin dashboard where System Administrators can Create, Read, Update, and Delete (CRUD) all attraction data to keep the system current.
Priority: HIGH

4.5.2 Stimulus/Response Sequences

- View All: Admin logs in- System shows all places with stats.
- Add: Admin fills the form and submits -System saves the new place.
- Edit: Admin modifies fields and submits - System updates the place.
- Delete: Admin clicks delete → System asks for confirmation and removes it.

4.5.3 Functional Requirements

- FR-21: The system shall provide a secure admin dashboard for authenticated Administrators only.
- FR-22: The system shall allow Admins to create a new place (name, category, description, location, distance, hours, fee, tips).
- FR-23: The system shall allow Admins to edit all fields of an existing place.
- FR-24: The system shall allow Admins to delete a place with a confirmation prompt.
- FR-25: The system shall support image uploads for places (JPG, PNG, GIF; max 5MB).
- FR-26: The system shall implement validation on all admin input forms.
- FR-27: The system shall log all admin actions for audit purposes.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-01: The system shall load the initial page within 3 seconds on a standard broadband connection.
- NFR-02: The system shall load detailed attraction information within 2 seconds.
- NFR-03: The system shall support at least 100 concurrent users without performance degradation.
- NFR-04: API endpoints shall respond within 500ms for read operations and 1 second for write operations.
- NFR-05: The Leaflet map with OpenStreetMap tiles shall render the map within 3 seconds.
- NFR-06: The system shall implement pagination (10 records per page).
- NFR-07: The system shall achieve a Google PageSpeed score of at least 80.
- NFR-08: The system shall maintain 99% uptime during operational hours.

5.2 Safety Requirements

- NFR-42: The system shall display a disclaimer stating that all information is for informational purposes only and users should exercise their own judgment when visiting locations.
- NFR-43: The system shall display emergency contact numbers (local police, hospital, tourism police) in a prominent location.
- NFR-44: The system shall include a "Safety Tips" section with general travel safety advice.
- NFR-45: The system shall display safety-related information for attractions where available (e.g., "Caution: Slippery rocks," "No lifeguard on duty").
- NFR-46: The system shall not store or process any sensitive personal information (payment details, medical information, etc.).

- NFR-47: The system shall not actively collect user location data without explicit consent.
- NFR-48: The system shall ensure all external links open in a new tab with appropriate warnings.
- NFR-49: The system shall have a process for reporting and correcting inaccurate or outdated information.
- NFR-50: The system shall not provide real-time emergency services but shall direct users to official emergency contacts.
- NFR-51: The system shall display appropriate warnings for attractions with safety risks (natural hazards, wildlife, weather conditions).
- NFR-52: The system shall include child safety information for family-friendly attractions.
- NFR-53: The system shall comply with Sri Lanka Tourism Development Authority (SLTDA) guidelines where applicable.

5.3 Security Requirements

- NFR-09: The system shall use HTTPS for all communications between client and server.
- NFR-10: The system shall encrypt all passwords using bcrypt with a salt factor of 10 before storing.
- NFR-11: The system shall use JWT (JSON Web Tokens) for authentication and session management.
- NFR-12: The system shall implement rate limiting (maximum 5 login attempts per 15 minutes) to prevent brute force attacks.
- NFR-13: The system shall protect against SQL injection, XSS, and CSRF attacks.
- NFR-14: The system shall store JWT tokens in HttpOnly cookies to prevent XSS token theft.
- NFR-15: The system shall expire sessions after 60 minutes of inactivity.

- NFR-16: The system shall use environment variables for all sensitive configuration (database credentials, JWT secret, API keys).

- NFR-17: The system shall validate all user inputs on both client and server sides.

- NFR-18: The system shall not expose sensitive information (stack traces) in error messages.

- NFR-19: The system shall enforce password strength: minimum 8 characters, at least one number, and one special character.

- NFR-20: The system shall implement password reset tokens that expire after 24 hours.

5.4 Software Quality Attributes

- NFR-21: The system shall recover gracefully from database connection failures and retry operations.

- NFR-22: The system shall implement error boundaries in React to prevent complete application crashes.

- NFR-23: The system shall follow MERN stack architecture with clear separation of concerns (frontend, backend, database).

- NFR-24: The system codebase shall be well-documented with comments and README files.

- NFR-25: The system shall use modular, reusable React components.

- NFR-26: The system shall follow a consistent coding style and naming convention.

- NFR-27: The system shall use environment-based configuration for development, staging, and production.

- NFR-28: The system shall be usable by users with no prior training or technical expertise.

- NFR-29: The system shall provide visual feedback for all user actions (loading indicators, success/error messages).

- NFR-30: The system shall run on all modern browsers (Chrome, Firefox, Safari, Edge).
- NFR-31: The system shall be deployable on any platform that supports Node.js and MongoDB.
- NFR-32: The system shall have unit tests for critical business logic and API endpoints.
- NFR-33: The system shall have integration tests for database operations and API flows.

5.5 Business Rules

- NFR-34: Only System Administrators can create, edit, or delete attraction data.
- NFR-35: Tourists can browse, search, view details, and build a session-based itinerary WITHOUT logging in.
- NFR-36: Only the "Save Itinerary" feature requires tourist authentication (login).
- NFR-37: The system shall not offer online booking, payment processing, or reservations for any services.
- NFR-38: All information displayed must be culturally sensitive and respect the diversity of communities in Akkaraipattu.
- NFR-39: The system shall include a disclaimer that it is an academic project and not an official government tourism website.
- NFR-40: The system administrator is responsible for ensuring the accuracy and currency of all information displayed.
- NFR-41: The system shall comply with applicable data protection regulations (GDPR, Sri Lanka Data Protection Act).

6. Other Requirements

Appendix A: Glossary

Table 3 Glossary of Terms and Abbreviations

Term	Definition
Akkaraipattu	Town in Ampara District, Eastern Province, Sri Lanka
API	Application Programming Interface
Bcrypt	Password hashing function for secure storage
CRUD	Create, Read, Update, Delete - basic database operations
CSRF	Cross-Site Request Forgery - security attack
CORS	Cross-Origin Resource Sharing
Express.js	Web framework for Node.js
HTTPS	Hypertext Transfer Protocol Secure
Itinerary	Planned route or journey schedule
JWT	JSON Web Token - authentication mechanism
MERN Stack	MongoDB, Express.js, React.js, Node.js
MongoDB	NoSQL document-oriented database
Node.js	JavaScript runtime environment
Postman	API testing tool
React.js	JavaScript library for user interfaces
REST API	Representational State Transfer API
SRS	Software Requirements Specification
SQL Injection	Code injection attack on databases
System Administrator	User who manages system content
Tourist User	Authenticated user with advanced features
XSS	Cross-Site Scripting - web vulnerability

Appendix B: Analysis Models

B.1 Entity Relationship Diagram

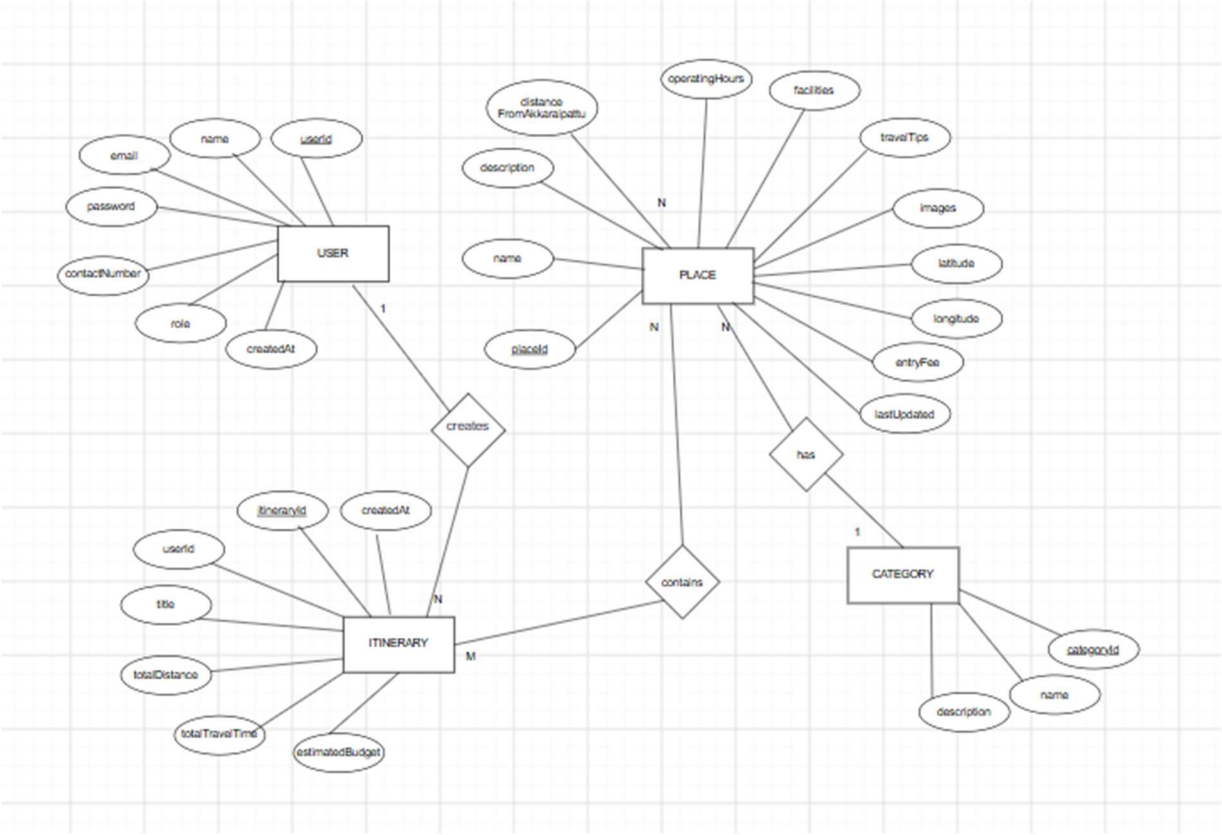


Figure i Entity-Relationship (ER) Diagram

B.2 Use Case Diagram

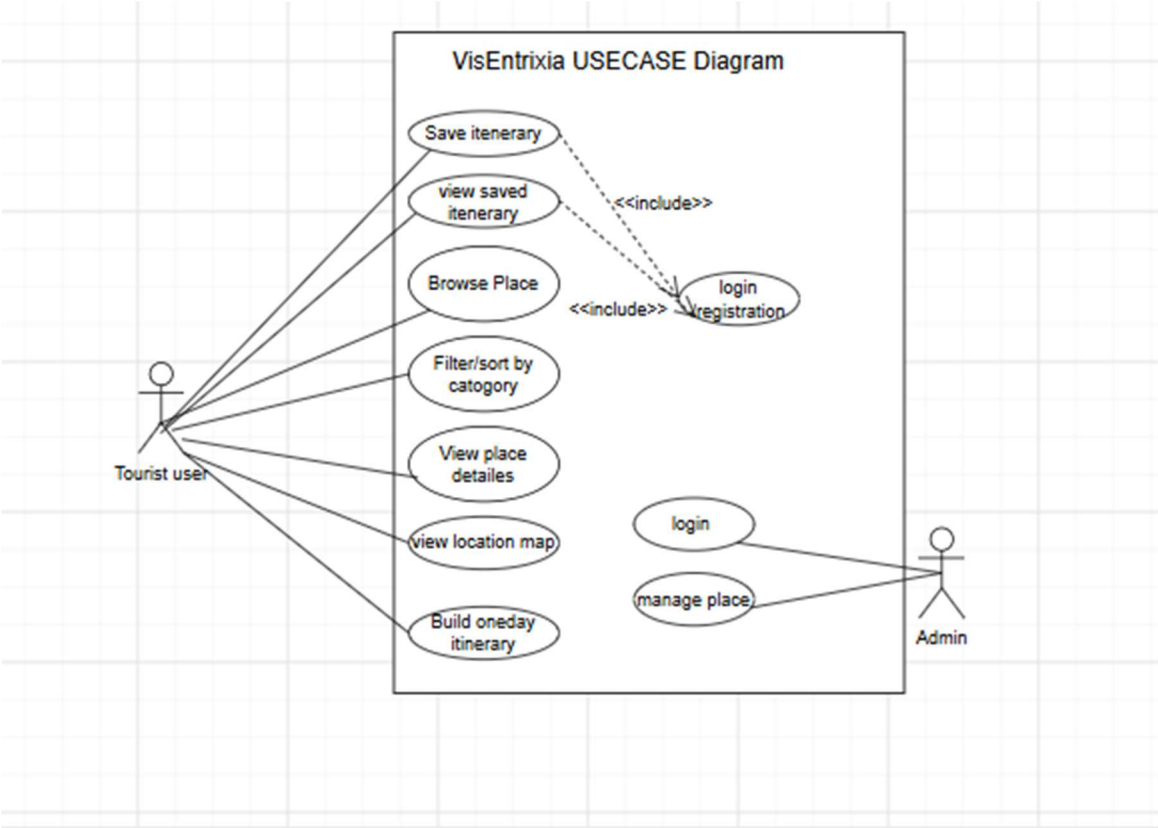


Figure ii Use Case Diagram

Appendix C: To Be Determined List

Table 4 To Be Determined List

TBD ID	Description
TBD-01	Hosting platform selection (AWS, Heroku, DigitalOcean)
TBD-02	Domain name selection for production deployment
TBD-03	Database backup and recovery procedures
TBD-04	End-user documentation (in-app help, user guide, tooltips)
TBD-05	Security audit schedule
TBD-06	Performance testing tools and metrics
TBD-07	SLTDA disclaimer or endorsement
TBD-08	Disaster recovery plan
TBD-09	Maintenance schedule and responsibilities
TBD-10	Accessibility compliance (WCAG 2.1 AA) verification plan